

Contents

- 1. [About the project](#)
- 2. [Workflow](#)
- 3. [Technology stack](#)
- 4. [OS-level data extraction](#)
- 5. [Functional scope and quality attributes](#)
- 6. [Endpoint agent design](#)

1. About the project

InfraPulse collects hardware, software, network and user inventory from Windows, macOS and Linux workstations across branch offices, and turns each collection into a **regulatory compliance report** — a formatted PDF and a schema-conformant XML.

It is built around a single deliverable: **a prescribed audit document**. That distinguishes it from general device-management platforms, which produce live dashboards rather than documents.

What makes it different

Agentless option	A workstation can be audited once, by running a launcher, with nothing left installed
Scheduled option	Or an installer registers a recurring audit via Task Scheduler / systemd / launchd
Fixed-format output	PDF + XML in the exact layout the audit mandate specifies
Asset lifecycle	Warranty, purchase order, supplier and vendor data that no endpoint agent can read
Zero mandatory infrastructure	PostgreSQL is optional — the system falls back to file storage and keeps working

Scale of the codebase

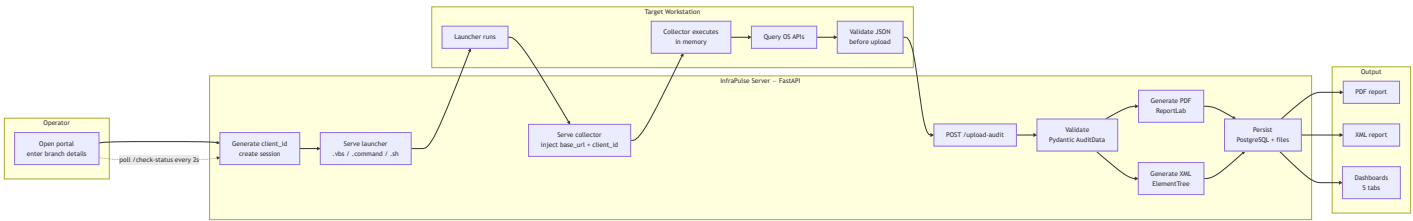
Component	File	Lines
Backend API + report engine	backend/main.py	3,017
Single-page web UI	frontend/index.html	3,666
macOS / Linux collector	scripts/audit.sh	2,186
Windows collector	scripts/audit.ps1	1,151
Installers (3 platforms)	installer/**	~800
Total		~10,800

Data captured

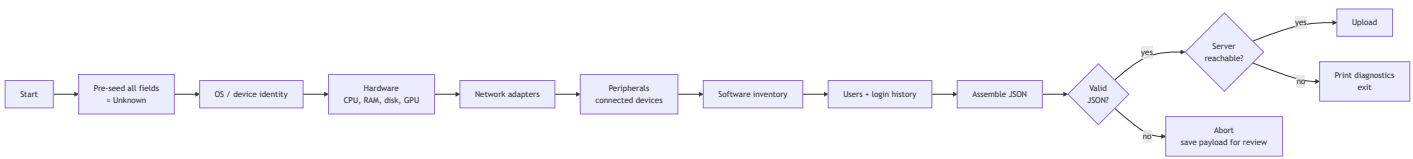
67 required fields across seven groups — Device Data (25), Disk Information (10), Partitions (5), Network Adaptors (12), Peripherals (4), Video Controllers (4), Users (7) — plus full software inventory, hotfix history, login history, antivirus state and a 17-field asset lifecycle record.

2. Workflow

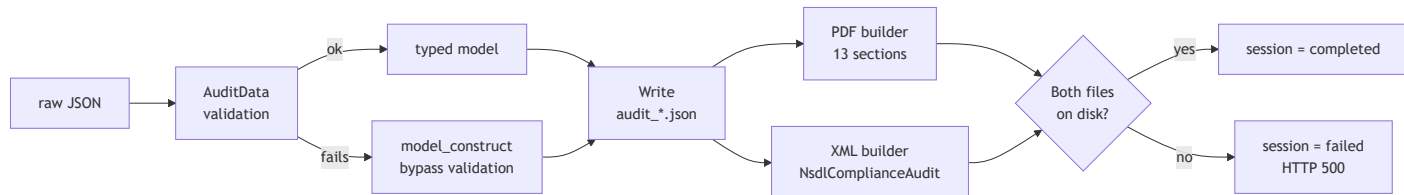
End-to-end flow



Collector internal sequence



Report generation



3. Technology stack

Backend

Layer	Technology	Role
Web framework	FastAPI	Async HTTP API, automatic OpenAPI docs
ASGI server	Uvicorn	Production server
Validation	Pydantic v2	<code>ConfigDict</code> , <code>field_validator</code> , typed payload models
Database	PostgreSQL via <code>psycopg2-binary</code>	<code>ThreadedConnectionPool</code> , <code>JSONB</code> columns
PDF generation	ReportLab	<code>SimpleDocTemplate</code> , <code>Table</code> , <code>Paragraph</code> , <code>KeepTogether</code>
XML generation	<code>xml.etree.ElementTree</code>	Standard library
Config	python-dotenv	<code>.env</code> loading
Concurrency	<code>concurrent.futures</code>	Thread pools for network scanning
Remote execution	pywinrm , pypsexec , paramiko , smbprotocol	Optional, lazily imported

Frontend

Layer	Technology
Structure	Single-page HTML5 , no framework
Logic	Vanilla JavaScript — <code>fetch</code> , 2-second polling
Styling	Hand-written CSS

Deliberately dependency-free — no build step, no `node_modules`, served directly by `StaticFiles`.

Endpoint collectors

Platform	Language	Runtime requirement
Windows	PowerShell 5.1+	Built into Windows
macOS / Linux	Bash + embedded Python 3	Both present by default on target systems

Scheduling

Platform	Mechanism
Windows	<code>schtasks</code> — hourly trigger + <code>ONSTART</code>
Linux	systemd timer , <code>cron</code> fallback
macOS	launchd LaunchDaemon with <code>StartInterval</code>

4. OS-level data extraction

How each platform is actually queried. **No kernel drivers, no privileged hooks** — every value comes from a documented operating-system interface.

4.1 Windows

Area	API / library	Notes
OS identity	<code>Get-CimInstance Win32_OperatingSystem</code>	Caption, version, architecture, <code>CSDVersion</code> for service pack
OS build name	Registry <code>HKLM:\SOFTWARE\Microsoft\Windows NT\CurrentVersion</code>	<code>DisplayVersion</code> + <code>CurrentBuild</code> + <code>UBR</code> → <code>25H2 (Build 26200.8894)</code> . WMI returns only the kernel version
Device type	<code>Win32_SystemEnclosure.ChassisTypes</code>	DMI code mapped via switch; falls back to <code>Win32_ComputerSystem.PCSystemType</code> ; VM detected by model regex
Licence status	<code>SoftwareLicensingProduct</code>	Filtered by Windows <code>ApplicationID</code> ; <code>LicenseStatus</code> 0–6 mapped to names
Hotfixes	<code>Get-HotFix</code>	Caption, HotFixID, description, install date

Area	API / library	Notes
System identity	<code>Win32_ComputerSystem</code>	Manufacturer, model, domain, domain role, total RAM
CPU	<code>Win32_Processor</code>	Name, core count, processor type
Memory slots	<code>Win32_PhysicalMemory</code>	Per-module capacity, slot count, max supported
BIOS	<code>Win32_BIOS</code>	Version and release date
Physical disks	<code>Win32_DiskDrive</code>	Model, serial, firmware, size, interface
SSD vs HDD	<code>MSFT_PhysicalDisk</code> in <code>root\Microsoft\Windows\Storage</code>	Authoritative — <code>MediaType</code> 3=HDD, 4=SSD, 5=SCM. Model-name regex used only as fallback
Disk → volume link	<code>Get-CimAssociatedInstance</code>	Walks <code>DiskDrive</code> → <code>DiskPartition</code> → <code>LogicalDisk</code> for file system and free space
Partitions	<code>Win32_LogicalDisk</code>	Device ID, file system, size, free space, boot flag
Optical drive	<code>Win32_CDROMDrive</code>	Absence reported explicitly, not assumed
GPU	<code>Win32_VideoController</code>	Name, driver version, <code>AdapterRAM</code> for VRAM
Network adapters	<code>Win32_NetworkAdapter</code> + <code>Win32_NetworkAdapterConfiguration</code>	MAC, speed, IPv4/IPv6, gateway, DNS, DHCP server, MTU
Primary MAC	<code>Get-NetAdapter</code>	First adapter with <code>Status -eq "Up"</code>
Printers	<code>Win32_Printer</code>	Name, system name, BIDI flag, port, extended status
Peripherals / devices	<code>Win32_PnPEntity</code>	Every plug-and-play device across USB, PCI, Bluetooth, serial, display
Antivirus	<code>AntiVirusProduct</code> in <code>root\SecurityCenter2</code>	The only reliable enumeration of third-party AV
Software inventory	Registry — 3 <code>Uninstall</code> hives	<code>HKLM</code> , <code>HKLM\WOW6432Node</code> , <code>HKCU</code> . Omitting <code>WOW6432Node</code> hides all 32-bit apps
Software last-used	<code>%SystemRoot%\Prefetch*.pf</code>	<code>LastWriteTime</code> of the prefetch file ≈ last run. Fuzzy name match; falls back to <code>install-dir mtime</code>

Area	API / library	Notes
User accounts	<code>Get-LocalUser</code>	Name, enabled state, home directory, last logon
Login history	<code>Get-WinEvent</code> — Security log, Event ID 4624	Requires Administrator. Parsed as XML; <code>LogonType</code> filtered to 2/7/10/11; machine accounts excluded
Uptime	<code>Win32_OperatingSystem.LastBootUpTime</code>	Converted to seconds + human display
Geolocation	HTTP → <code>ip-api.com/json/</code>	Public IP, city, region, country. The only outbound call

4.2 Linux

Area	API / library	Notes
OS identity	<code>/etc/os-release</code> , <code>uname -s -r -m</code>	<code>NAME</code> , <code>VERSION_ID</code> , <code>PRETTY_NAME</code> , kernel release
Hardware identity	<code>/sys/class/dmi/id/*</code>	SMBIOS via sysfs — vendor, product name, serial, chassis type, BIOS version/date. Same firmware tables WMI reads
Device type	<code>/sys/class/dmi/id/chassis_type</code>	Identical numbering to Windows <code>ChassisTypes</code> — the switch table is shared
Container detection	<code>/.dockerenv</code> , <code>/proc/1/cgroup</code>	Matches <code>docker</code> <code>lxc</code> <code>kubepods</code>
VM detection	<code>systemd-detect-virt</code>	Reports the specific hypervisor
CPU	<code>/proc/cpuinfo</code> , <code>lscpu</code>	Model name, core and socket counts
Memory	<code>/proc/meminfo</code> , <code>dmidecode</code>	Total memory; DMI for slot count and max capacity
Uptime	<code>/proc/uptime</code>	Seconds since boot
Block devices	<code>/sys/block/*</code>	<code>sr[0-9]*</code> entries are optical drives
SSD vs HDD	<code>/sys/block/*/queue/rotational</code>	Kernel's own flag — <code>0</code> = SSD, <code>1</code> = HDD. Not an inference
Disks / partitions	<code>lsblk</code> , <code>df</code>	Size, mount point, file system, free space

Area	API / library	Notes
GPU	<code>lspci -k</code>	<code>-k</code> reports the kernel driver actually bound , not merely installed
USB devices	<code>lsusb</code>	Vendor, product, bus and device number
PCI devices	<code>lspci</code>	Class, vendor, device
Network interfaces	<code>ip addr</code> , <code>ip link</code> , <code>ifconfig</code>	MAC, IPv4/IPv6, MTU, state
Network connection	<code>nmcli -t -f ...</code>	Active connection, SSID. Terse mode escapes colons as <code>\:</code>
Routing / gateway	<code>ip route</code>	Default gateway
DNS	<code>/etc/resolv.conf</code>	Nameservers, search domain
Software inventory	<code>dpkg-query -W</code> , <code>rpm -qa</code>	Debian first; rpm runs only if dpkg returned nothing
Software last-used	<code>shutil.which()</code> + <code>os.path.getatime()</code>	Access time of the resolved binary. Unreliable under <code>noatime/relatime</code>
Users	<code>getent passwd</code> , <code>/etc/shadow</code>	Name, home directory, shell, account state
Login history	<code>last</code> , <code>lastlog</code>	Login and shutdown records
Printers	<code>lpstat</code> , CUPS	

4.3 macOS

Area	API / library	Notes
OS identity	<code>sw_vers -productVersion</code> , <code>-buildVersion</code>	Product version and build
Hardware model	<code>sysctl -n hw.model</code>	Model identifier, e.g. <code>MacBookPro18,3</code>
Device type	<code>hw.model</code> string match	<code>*Book*</code> → Laptop, <code>*iMac*</code> → All-in-One, <code>*mini*</code> → Desktop
CPU / RAM	<code>sysctl -n hw.ncpu</code> , <code>hw.memsize</code>	Core count, physical memory bytes
System identity	<code>system_profiler SPHardwareDataType</code>	Serial number, model name, chip, memory

Area	API / library	Notes
Software inventory	<code>system_profiler SPApplicationsDataType -json</code>	Native JSON output — parsed directly, no text scraping
Software last-used	<code>os.path.getatime()</code> on the <code>.app</code> bundle	Falls back to <code>lastModified</code> from <code>system_profiler</code>
GPU	<code>system_profiler SPDisplaysDataType</code>	Chipset, VRAM, resolution
Storage	<code>system_profiler SPStorageDataType, diskutil</code>	Volumes, size, free space, file system
USB devices	<code>system_profiler SPUSBDataType</code>	Device tree with vendor and product
Optical drive	<code>system_profiler SPDiscBurningDataType, drutil status</code>	Two sources tried in order
Device tree	<code>ioreg</code>	IORegistry for low-level device data
Network interfaces	<code>ifconfig, networksetup</code>	MAC, addresses, hardware ports
Wi-Fi / IP	<code>ipconfig getifaddr en0</code> then <code>en1</code>	Adapter name varies by model
Users	<code>dscl . -list /Users, dscacheutil</code>	Directory Services
Login history	<code>last</code>	
Printers	<code>lpstat -p</code> , CUPS	

4.4 Extraction design notes

- Why `system_profiler -json` matters.** macOS is the only platform with a native structured inventory. Its collectors hand output straight to `json.loads()`, making the macOS branches consistently shorter and less fragile than the Linux equivalents, which parse text.
- Why Linux and Windows share a chassis table.** Both ultimately read the same **SMBIOS/DMI tables** published by system firmware — Windows through `Win32_SystemEnclosure`, Linux through `/sys/class/dmi/id/`. The numeric codes are identical, so one switch statement serves both.
- Authoritative source before heuristic.** Wherever the OS *knows* an answer, that source is queried first and inference is the fallback — `MSFT_PhysicalDisk.MediaType` and `queue/rotational` over model-name matching; `ChassisTypes` over guessing; Security event log over `LastLogon`.

5. Functional scope and quality attributes

5.1 Functional scope

#	Capability	Description	Status
F1	Compliance audit	Launcher distribution, collection, PDF + XML generation, status polling, download	Complete
F2	Asset registry	CRUD over 17 lifecycle fields — owner, warranty, PO, supplier, vendor, status	Complete
F3	Device audits	Per-device hardware specs, software inventory, incremental change tagging	Complete
F4	Change reporting	Diff of the two most recent audits — hardware deltas, installed/removed software	Complete
F5	Network discovery	TCP port scan over a CIDR, ARP fallback, four-source name enrichment	Complete — LAN-bound
F6	Wi-Fi dashboard	Scan networks, connect, enumerate subnet devices	Windows-centric
F7	Scheduled re-audit	Recurring collection via native OS schedulers	Complete
F8	Remote audit trigger	WinRM / PsExec / SSH initiation	Partial — stub
F9	Authentication	—	Not implemented
F10	Vulnerability management	—	Not in scope

5.2 Quality attributes

Attribute	Design approach	Current state
Reliability	Pre-seeded fallbacks + try/except on every collector; a failed field never aborts the audit. PDF and XML generated independently	Strong. The collector cannot crash on a collection failure
Data integrity	Five validation layers: pre-seed → normalise → authoritative source → structural JSON validation → server-side Pydantic	Strong on structure. No cross-source reconciliation or attestation
Availability	PostgreSQL optional with a complete file-storage fallback; DB outage degrades rather than fails	Strong

Attribute	Design approach	Current state
Portability	Three collectors covering Windows, macOS, Linux; shared DMI code table; tool-availability guards before every external command	Strong
Usability	Single-page UI, no build step; double-click launchers; self-elevating installers; four numbered diagnostics on connection failure	Strong
Performance	Wide thread pools (up to 512 workers) for scanning; reverse DNS capped at 3s; /24 scan completes in seconds	Good for LAN scale
Scalability	—	Weak. In-memory sessions dict forces -workers 1; audit_results grows unbounded; fallback paths do O(n) directory scans. Practical ceiling ~200–500 hosts
Security	—	Weak. No authentication on any endpoint; HTTP by default; no TLS 1.2 enforcement in the Windows collector; Wi-Fi passwords stored in plaintext; curl bash install channel
Maintainability	Section-banner organisation; extensive bug post-mortem comments	Mixed. ~10,800 lines with duplicated endpoints and repeated report-table blocks
Observability	Dual logging to file and stdout; per-step console output during collection	Adequate. 19 of 54 exception handlers are silent pass
Extensibility	extra="allow" on every Pydantic model — new fields accepted without schema changes or breaking installed agents	Strong
Auditability	Every audit persisted as raw JSON alongside the generated PDF and XML	Strong for retention, weak for provenance

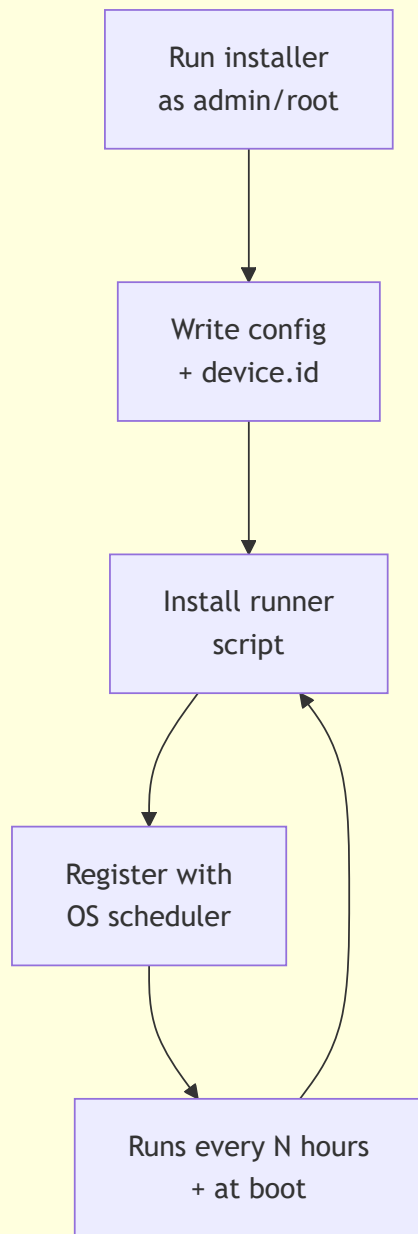
5.3 Known constraints

1. **F5 and F6 require the server on the target LAN** — they inspect the network the server is attached to, which blocks central cloud deployment of those two features.
 2. **Software inventory is truncated to 150 entries** on macOS and Linux; Windows is uncapped.
 3. **Login history requires Administrator** on Windows; otherwise a weaker fallback is used silently.
 4. **"Unknown" is ambiguous** — it means both "absent" and "collection failed".
-

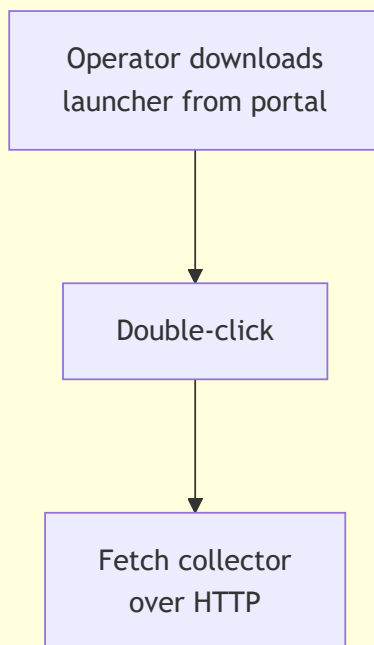
6. Endpoint agent design

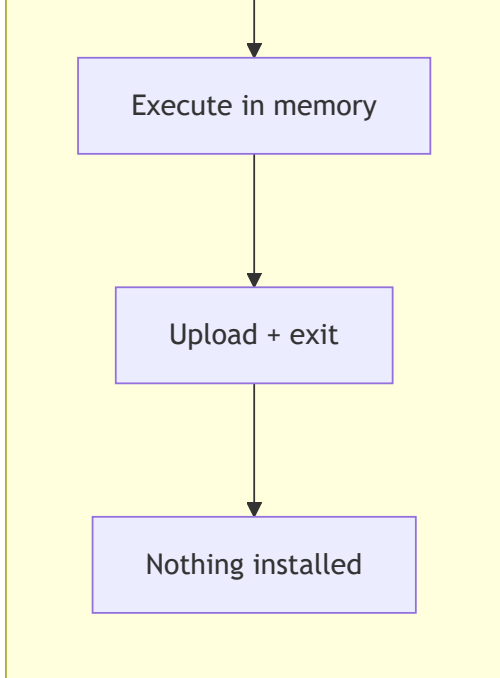
InfraPulse supports **two deployment modes** from the same collector code.

Mode 2 – Scheduled



Mode 1 – One-shot





6.1 Launcher artefacts

Platform	Artefact	Served by	Mechanism
Windows	<code>verify_system_<id>.vbs</code>	<code>/download-vbs</code>	<code>WScript.Shell</code> runs PowerShell hidden: <code>Invoke-RestMethod ... Invoke-Expression</code>
macOS	<code>verify_system_<id>.command</code>	<code>/download-mac</code>	<code>curl -s "..."</code> <code>bash</code> — <code>.command</code> is double-clickable in Finder
Linux	<code>verify_system_<id>.sh</code>	<code>/download-linux</code>	<code>curl -s "..."</code> <code>bash</code>

There is no compiled `.exe`. The Windows path uses a `.vbs` wrapper — chosen because double-clicking it runs PowerShell with **no visible console window** (`objShell.Run command, 0, False`). The installer entry point is `Install-Audit.bat`, which self-elevates:

```
net session >nul 2>&1
if %errorlevel%==0 goto RUN
powershell -NoProfile -Command "Start-Process -FilePath '%~f0' -Verb RunAs"
```

6.2 Key design property — the collector never touches disk

In one-shot mode the collector is fetched and executed **entirely in memory**:

```
.vbs -> powershell -ExecutionPolicy Bypass -WindowStyle Hidden
      -> Invoke-RestMethod '<server>/download-script?client_id=<id>'
```

The server substitutes `base_url` and `CLIENT_ID_PLACEHOLDER` into the script text at serve time, so the collector always calls back on the host the client actually used. Nothing is written to the workstation, nothing needs uninstalling.

6.3 Scheduled mode

The installers (`installer/{windows,macos,linux}/`) read a shared `config.txt`:

Setting	Purpose
<code>SERVER_URL</code>	Server address — must be the network IP, not localhost
<code>INTERVAL_HOURS</code>	Audit frequency
<code>INTERVAL_MINUTES</code>	Testing override
<code>JITTER_SECONDS</code>	Random pre-run delay so a branch does not hit the server simultaneously

Each installer:

1. **Verifies elevation** — Administrator / root / `sudo`, and refuses otherwise
2. **Checks prerequisites** — `curl`, `python3`
3. **Tests reachability** — `curl -s -f -m 10 "$SERVER_URL/api/devices"`
4. **Generates a stable `device.id`** — reused across runs so audits correlate to one machine
5. **Writes a small runner script** — deliberately minimal; it only fetches and executes the current collector, so server-side collector updates apply without revisiting the PC
6. **Registers the schedule**

Platform	Registration	Boot trigger
Windows	<code>schtasks /SC HOURLY /MO <n> /RU SYSTEM /RL HIGHEST</code>	<code>/SC ONSTART /DELAY 0005:00</code>
Linux	<code>systemd timer, cron fallback to /etc/cron.d/</code>	<code>@reboot sleep 300</code>
macOS	<code>LaunchDaemon .plist with StartInterval</code>	<code>RunAtLoad</code>

Jitter is applied before each scheduled run:

```
WAIT=$(( $(od -An -N2 -tu2 < /dev/urandom | tr -d ' ') % JITTER ))
sleep "$WAIT"
```

Without it, every PC in a branch uploads at the same second after a power restoration.

6.4 Agent execution contract

Property	Behaviour
Privileges	Elevation required to <i>install</i> a schedule. One-shot mode runs unelevated, but login history then falls back to a weaker source
Failure mode	Degrade, never abort — a failed collector yields "Unknown" for its fields only
Pre-upload validation	<code>audit.sh</code> validates each JSON fragment and the whole payload; refuses to upload if invalid and saves it to <code>/tmp/audit_payload_invalid.json</code> . <code>audit.ps1</code> relies on <code>ConvertTo-Json -Depth 8</code>
Connectivity check	<code>audit.sh</code> tests TCP reachability via <code>nc</code> or <code>bash /dev/tcp</code> before uploading, printing four numbered diagnostics on failure
Upload	<code>audit.sh</code> : <code>curl</code> (60s cap) → <code>wget</code> fallback. <code>audit.ps1</code> : <code>Invoke-RestMethod</code> (300s) → <code>System.Net.WebClient</code> fallback
Idempotency	Every run produces a new audit record; the backend computes diffs against the previous one
Uninstall	<code>uninstall-audit.{ps1,sh,command}</code> removes the schedule, runner and config

6.5 Known agent-side gaps

1. **No TLS 1.2 enforcement in `audit.ps1`**. Windows PowerShell 5.1 defaults to TLS 1.0, so both upload attempts fail against a modern HTTPS endpoint.
2. `curl` | `bash` and `Invoke-RestMethod` | `Invoke-Expression` are the install channel — acceptable on a trusted LAN, a remote-code-execution path on a public URL.
3. **No payload signing or machine attestation** — `computer_name` is self-reported and `client_id` is a query-string parameter.
4. `audit.ps1` performs no pre-upload validation, unlike `audit.sh`.

Related documents

Document	Contents
CODEBASE_EXPLANATION.md	Line-by-line walkthrough of the backend, both collectors, and the verification design
tools-comp.md	InfraPulse vs osquery + Fleet — cost, security, scale, maintenance
data-fields-comparison.md	All 67 required fields mapped to the osquery schema; hybrid migration plan
WORKFLOW.md	Operational workflow

Document	Contents
README.md	Setup and running instructions